

Cairo University
Faculty of Engineering
Computer Engineering Department

Data Structures and Algorithms

Sheet #4 – Stacks and Queues

Part I: Tracing

pushStack(S, X)	Pushes a value X at the top of the stack S
popStack(S, X) Or X = pop(s)	Removes and returns the value at the top of the stack S into X
emptyStack(S) Or isempty(S)	Returns true if stack S is empty
X=StackTop(S) Or X=peek(S)	Returns (without removing) the value at the top of the stack S into X
enqueue(Q, X)	Enqueues a value X at the rear of the queue Q
dequeue(Q, X) Or X = dequeue(Q)	Removes and returns the value at the front of the queue Q into X
emptyQueue(Q) Or isempty(Q)	Returns true if queue Q is empty
X = Front(Q) Or X=Peek(Q)	Returns (without removing) the value at the front of the queue Q into X
loop(not end of file)	Keeps looping until all the give data are read
read a number	Reads on number from the given data

- Trace the algorithm and show the contents of the stack after each operation.

Algorithm: SimplePushPop()
Stack S ← empty //create empty Stack S
push(S, 10)
push(S, 20)
pop(S)
push(S, 30)
push(S, 40)
pop(S)
push(S, 50)

2. Consider the following algorithm. Trace each step and show the stack contents after each operation. Assume stack is passed as **bottom[8, 12, 20, 15] top**
- What are the stack contents after the algorithm completes?
 - What is the final value of c ?
 - What does this algorithm do ?

```

Algorithm: Algo(S)
    while not isEmpty(S)
        pop(S)
         $c \leftarrow c + 1$ 
    end while

```

3. The following algorithm is supposed to reverse stack S using a temporary stack Temp. Assume S is passed as **bottom [3, 6, 9] top**
- Show the contents of both stacks after each operation.
 - What is the content of the stack S at the end of the algorithm?
 - Has the algorithm reversed the stack S? If not, suggest a fix.

```

Rev (Stack S)
    Stack Temp1  $\leftarrow$  empty , Temp2  $\leftarrow$  empty
    while not isEmpty(S)
        pop(S, item)
        push(Temp1, item)
    end while

    while not isEmpty(Temp1)
        pop(Temp1, item)
        push(S, item)
    end while

```

4. Trace the algorithm to determine whether parentheses in a string **str** are balanced or not. Show the stack contents after each character is processed.

Test case: str = "((()))"

- Show the contents of the stack S at each step.
- What is the output of the algorithm for this test case?

```

Algorithm: IsBalanced(str)
    Stack S  $\leftarrow$  empty

    for each character in str do
        if character = '(' then push(S, character)
        else
            if character = ')' then
                if isEmpty(S) then return "Not Balanced"
            pop(S)
        end if
    end for

    if isEmpty(S) then return "Balanced"
    else return "Not Balanced"

```

5. Trace the following algorithm and show the contents of the Queue after each operation.

Algorithm: SimpleEnqueueDequeue()

```
Queue Q ← empty
enqueue(Q, 5)
enqueue(Q, 10)
dequeue(Q)
enqueue(Q, 15)
enqueue(Q, 20)
dequeue(Q)
enqueue(Q, 25)
```

6. The following algorithm is supposed to reverse Queue Q using a temporary queue Temp. Assume Q is passed as **front [1, 2, 3] rear**
- Show the contents of both queues after each operation.
 - What is the content of Q at the end of the algorithm?
 - Has the algorithm reversed Q? If not, suggest a fix.

Algorithm: ReverseQueue(Q)

```
Queue Temp ← empty
Stack S ← empty

while not isEmpty(Q)
    dequeue(Q, item)
    enqueue(Temp, item)
end while

while not isEmpty(Temp)
    dequeue(Temp, item)
    enqueue(Q, item)
end while
```

7. What are the contents of S, Q1, and Q2 after completion of the following algorithm

```
1 S = createStack
2 Q1 = createQueue
3 Q2 = createQueue
4 enqueue (Q1, 5)
5 enqueue (Q1, 6)
6 enqueue (Q1, 9)
7 enqueue (Q1, 0)
8 enqueue (Q1, 7)
9 enqueue (Q1, 5)
10 enqueue (Q1, 0)
11 enqueue (Q1, 2)
12 enqueue (Q1, 6)
13 loop (not emptyQueue (Q1))
  1 dequeue (Q1, x)
  2 if (x == 0)
    1 z = 0
    2 loop (not emptyStack (S))
      1 popStack (S, y)
      2 z = z + y
    3 end loop
    4 enqueue (Q2, z)
  3 else
    1 pushStack (S, x)
    4 end if
14 end loop
```

8. What are the contents of S1, and Q1 after completion of the following algorithm

```
1 Q1 = createQueue
2 S1 = createStack
3 loop (not end of file)
  1 read number
  2 if (number not 0)
    1 pushStack (S1, number)
  3 else
    1 popStack (S1, x)
    2 popStack (S1, x)
    3 loop (not empty S1)
      1 popStack (S1, x)
      2 enqueue (Q1, x)
    4 end loop
  4 end if
4 end loop
```

The data are 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0.

Part II: Applications

- 1- Match the following applications with the most appropriate data structure from the ones you've covered (array, linked list, stack, queue, priority queue):
 - a. **Social Media Feed:** This application displays a constantly updating list of posts from users you follow. New posts are added at the top, and older ones are scrolled out of view.
 - b. **Search Engine Results:** When you enter a search query, the engine returns a ranked list of webpages. The most relevant pages appear at the beginning of the list.
 - c. **Online Chat Application:** This application displays a real-time conversation between two or more users. New messages are added to the end of the conversation.
 - d. **Browser Back and Forward buttons:** Users can navigate back and forth through their browsing history.
 - e. **Music Playlist:** This application allows users to create a list of songs they want to listen to in **order**. Users can remove songs at any point.

2- Stack-based App: Backtracking

One of the applications of a stack is to **backtrack**—that is, to retrace its steps.

As an example, imagine we read a list of items, such that:

- If a positive number is encountered just push it into the stack.
- Each time we read a negative number we must backtrack and print the **five** numbers that come before the negative number (starting from the one previous to the negative number) and then discard the negative number.
- If there are fewer than **five** items in the stack, print an error message and stop the program.
- The program stops normally without an error message when zero is entered.

Trace the above algorithm using the following data. Show the contents and the output screen

1 2 3 4 5 -1 1 2 3 4 5 6 7 8 9 10 -2 11 12 -3 1 2 3 4 5

3- Queue-based App : Bank Queueing System

Assume we have a bank with **one teller** working only at certain interval of time. Normally the arrival and service time for each customer should follow some pattern (usually random distribution) but for simplicity, here is the table of customer arrival and service times for each customer.

Name	Arrival Time	Service Time	Service Start Time	Service End Time	Waiting Time
A	3	12			
B	9	2			
C	16	8			
D	30	20			
E	34	10			
F	45	9			
G	47	5			

Table 1

Requirements:

- 1- Extend and fill the following table that shows the contents of the customers waiting queue with each enqueue or dequeue operation illustrating the time of the operation beside the queue.

Time	Operation	Waiting Queue content. Assume Front is on the left
0	Empty Queue	[]
3	Enqueue cust A	[A]
3	Dequeue A	[]

Table 2

- 2- Fill Table1 for each customer
- 3- Calculate average waiting time
- 4- Repeat the above problem but with two tellers in the bank and compare the new average waiting time to the old one

Part III: Coding Problems

- Problems in this part are for self-training to practice coding
- Not to be solved during the problem set session
- You may try them during lab sessions
- Answers will be provided to you for guidance

- 1- Implement two stacks using only one array of int. Your stack algorithm should not declare an overflow unless every slot in the array is used. You should provide 2 functions for each stack operation (e.g. Push1: pushes in stack1, Push2: pushes in stack2) and so on.

```
class doubleStack{
    enum { maxSize = 100; }
    int arr[maxSize];
    int top1, top2;
public:
    doubleStack(.....); //constructor
    ..... push1(.....);
    ..... Push2(.....);
    bool isFull( );
    //and so on
};
```

- 2- Implement queue operations using two stacks. Keeping the **FIFO** property of the queue. Assume class mySack is fully implemented with all usual stack operations

```
class myQueue{
    myStack S1;
    myStack S2;
public:
    myQueue(.....); //constructor
    ..... enqueue(.....);
    ..... dequeue(.....);
    //and so on
};
```